CS3230 : Design and Analysis of Algorithms (Spring 2014)

Tutorial Set #8

[For discussion during Week 10]

OUT: 20-Mar-2014 **Tutorials:** Mon & Thu, 24, 27 Mar 2014

Prepare your answers to all the D-Problems in every tutorial set.

When preparing to present your answers,

- Think of a CLEAR EXPLANATION
- Illustrate with a good worked example;
- Describe the main ideas,
- Can you sketch why the solution works;
- Give analysis of running time, if appropriate
- Can you think of other (perhaps simpler) solutions?

In CS3230, you learn to develop high-level abstractions when describing algorithms. Try not to speak in ML/AL (machine/assembly language) or "for (j=0; j<n; j++) do". Instead give names to your sets (of objects or things or data structures), talk about depth-first search, binary search, traverse the graph, sort the set, use a priority queue, etc. You are no longer in CS1010, CS1020, CS2010 or CS2020. Speak with greater sophistication, and at a higher level of abstraction.

***Routine Practice Problems** -- do not turn these in -- but make sure you know how to do them.*

R1. (Exercises 22.1-1 of [CLRS]) [Degree of Vertices]
Given an adjacency-list representation of a directed graph, how long does it take to compute the out-degree of every vertex? How long does it take to compute the in-degrees?

R2. (Exercises 22.1-2 of [CLRS]) [Graph DS]
Give an adjacency-list representation for a complete binary tree on 7 vertices. Give an equivalent adjacency-matrix representation. Assume that vertices are numbered from 1 to 7 as in a binary heap.

R3. (Exercise 22.3-10 of [CLRS]) [DFS]
Modify the pseudocode for depth-first search so that it prints out every edge in the directed graph G, together with its type. Show what modifications, if any, you need to make if G is undirected.

Answers:
R1: Out-degree: O(V) if each vertex remembers the size of its neighbor, or O(V+E) otherwise.
In-degree: Can be done in O(V+E), scan through all edges once and increment the in-degree counter for each vertex.

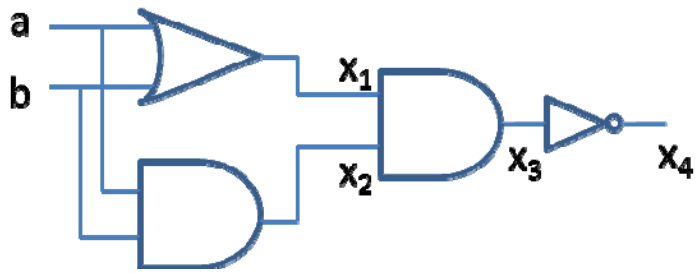
R2: Try directly at http://www.comp.nus.edu.sg/~stevenha/visualization/graphds.html

R3: See page 129 of CP3, or open file (ch4_04_dfs.cpp/java) in ch4.zip available at https://sites.google.com/site/stevenhalim/home/material

Discussion-Problems -- Solve these problems and present them in tutorial class.

D1. (Lecture08 review, reduction from Circuit-SAT to SAT problem)

Consider the following Boolean circuit for the Circuit-SAT.
Can you transform it to the corresponding Boolean function for the SAT problem using **both** the incorrect (non-polynomial) reduction and the correct (polynomial) reduction as shown in Lecture 08?



Solution:

For this small circuit, the incorrect (non-polynomial) reduction shown in class does not really show us the exponential number of variables:
$\phi_1 = x_4 = \neg x_3 = \neg(x_1 \wedge x_2) = \neg((a \vee b) \wedge (a \wedge b))$.

The correct reduction looks like it generates more complex formula, but this is polynomial:
$\phi_2 = x_4 \wedge (x_4 \leftrightarrow \neg x_3) \wedge (x_3 \leftrightarrow (x_1 \wedge x_2)) \wedge (x_2 \leftrightarrow (a \wedge b)) \wedge (x_1 \leftrightarrow (a \vee b))$.

Follow up question: Why do we have output variable (x_4 in this example) $\wedge$ (operation for each gate) in the correct reduction?
Answer: This is to ensure that the reduction is correct. If you take the output variable out from the correct reduction, you will not get a zero (unsatisfiable) in the reduced formula ϕ even if the original circuit is unsatisfiable.

D2. (Lecture 08+09 review)

a. Prove that the Longest-Path problem is NP-Complete assuming that you already know that another problem Hamiltonian-Path is in NP-Complete.

Formal definition of Longest-Path: Given an undirected graph G = (V,E) and a positive integer $K \leq |V|-1$, does G contain a simple path (that visits each vertex once) with <u>K or more</u> edges?

Formal definition of Hamiltonian-Path: Given a graph G = (V,E), does it have a Hamilton path (a path that visits each vertex exactly once)?

Solution:

As usual, we use the two steps approach

1. Show that Longest-Path problem is in NP.
   This is quite trivial, given a certificate (the longest path itself), the verifier algorithm traverses the path to verify if each consecutive vertices are in E, and checks if all vertices in G are visited only once, and the path contains K or more edges.

2. Show that Hamiltonian-Path $\leq_p$ Longest-Path
   In this problem, we are given a fact that Hamiltonian-Path has been proven to be NP-Complete. These two problems are related, and the polynomial-time reduction is easy.

Given a Hamiltonian-Path instance, we can simply set K = |V|-1. If G(V,E) has a Hamiltonian-Path, it has a Longest-Path of K = |V|-1 edges and vice versa.
This is clearly an O(1), polynomial transformation.

b. Show a polynomial time algorithm that can decide Eulerian-Path and Eulerian-Cycle problems to show that both Eulerian-Path and Eulerian-Cycle problems are in P.

Formal definition of Eulerian-Path: Given a graph G = (V,E), does it have an Eulerian path (a path that visits each edge exactly once)?

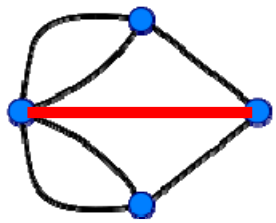
PS: Do you notice that the problem definition is 99% similar to Hamiltonian-Path?

Formal definition of Eulerian-Cycle: Given a graph G = (V,E), does it have an Eulerian path (a path that visits each edge exactly once) and the first vertex is equals to the last vertex?

Solution:

For Eulerian path, the algorithm simply checks if all vertices in G except two have even degrees and only two vertices have odd degrees (For Eulerian cycle, all vertices must have even degrees). Depending on the way the graph G is stored, such algorithm either runs in O(V), O(V+E), or even $O(V^2)$ but all these are still polynomial.

This algorithm is based on the work by Leonard Euler in 1700ies who show that the Seven bridges of Konigsberg does not have Eulerian cycle.



Note: If we delete the **thick red edge** from the graph above, we can have an Eulerian path. We start at one vertex with odd degree 3 and ends at another vertex at odd degree 3.

Discussion point: A similarly worded problem can have a totally different solution (Shortest vs Longest-Path, Eulerian-Cycle vs Hamiltonian-Cycle, 2-CNF-SAT (or DNF-SAT) vs 3-CNF-SAT, Bitonic-TSP vs TSP).

D3. (Lecture 09 review, Reduction Exercise again, designed to help students who are still struggling with PA2 subtask E)
Draw the implication graph of the following 2-CNF-SAT instances below using the same rules as in PA2 subtask E.
$\phi = (\neg x_4 \vee x_1) \wedge (x_3 \vee x_1) \wedge (\neg x_1 \vee x_1) \wedge (x_3 \vee x_1) \wedge (x_3 \vee x_3)$

a. What are the vertex numbers of these variables (and its negation) in ϕ according to PA2 subtask E rule:, x_1, $\neg x_1$, x_3, $\neg x_3$, x_4, $\neg x_4$?
b. What is the number of vertices of G that have to be reported in PA2 subtask E?
c. The third clause is $(\neg x_1 \vee x_1)$. Will this clause generate any edge in G according to PA2 subtask E rule?

d. The second and the fourth clauses are the same. Will this causes multi edges according to PA2 subtask E rule?

e. The fifth clause is ($x_3 \vee x_3$). Will this clause generate any edge in G according to PA2 subtask E rule?

f. Now draw the graph G and report the number of edges in G!

g. List down V, E, and E edges according to PA2 subtask E format!

h. Is ϕ satisfiable? Visually verify whether there is a variable and its negation in the same SCC of G.

Solution:

a. x_1 will have vertex number: 2*(1-1)+0 = 0
$\neg x_1$ will have vertex number: 2*(1-1)+1 = 1
x_3 will have vertex number: 2*(3-1)+0 = 4
$\neg x_3$ will have vertex number: 2*(3-1)+1 = 5
x_4 will have vertex number: 2*(4-1)+1 = 6
$\neg x_4$ will have vertex number: 2*(4-1)+1 = 7

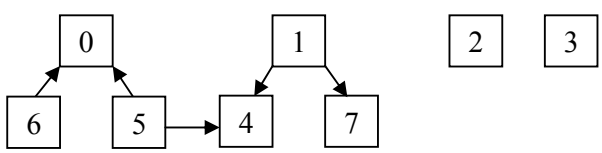
b. $\neg x_4$ is the variable with the largest variable number, not x_3. Therefore we need to create up to 2*4 = 8 vertices in G. An incorrect implementation may accidentally take x_3, i.e. the positive x_3, as the largest variable number. Another wrong implementation may report 4 (or 6) vertices as only 4 different literals (6 different vertices) are used in the formula (in graph G), skipping x_2. We need to standardize the graph G construction process for automated judging.

c. ($\neg x_1 \vee x_1$) is equivalent to $x_1 \rightarrow x_1$ and $\neg x_1 \rightarrow \neg x_1$. Both are self-loop edges and they violate the condition that the two endpoints have to be different in PA2 subtask E. Therefore these two self-loop edges are not added to G.

d. ($x_3 \vee x_1$) is equivalent to $\neg x_3 \rightarrow x_1$ (edge 5→0) and $\neg x_1 \rightarrow x_3$ (edge 1→4). We prevent multi edges in PA2 subtask E, therefore only one copy of each duplicated edges can be included in G.

e. ($x_3 \vee x_3$) is equivalent to $\neg x_3 \rightarrow x_3$ (edge 5→4) and the same $\neg x_3 \rightarrow x_3$. The first edge is taken, but the second duplicated edge is excluded.

f. The graph G with 4 edges is shown below.



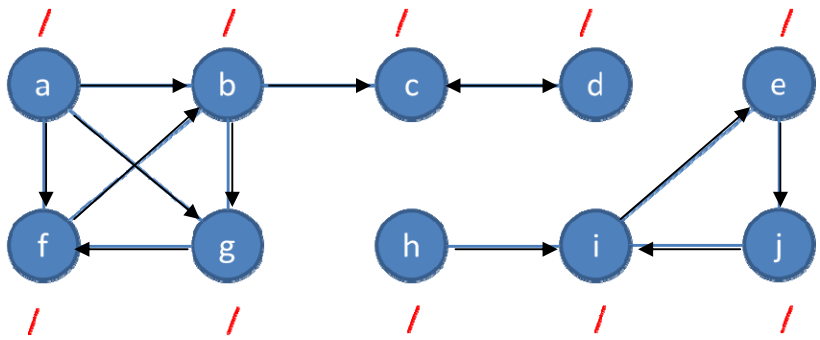
g. The output format according to PS2 subtask E for this graph G (excluding the number of test cases I) is as follows:

8 5
1 4
1 7
5 0
5 4
6 0

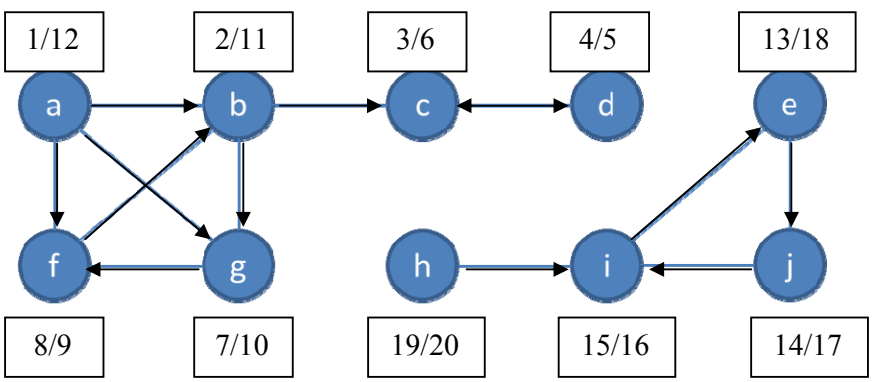
h. It is satisfiable. No variable and its negation belong to the same SCC of G. (There are 8 SCCs in this G, i.e. each vertex is an SCC).

D4. (Lecture09 review, designed to help students who are still struggling with PA2 subtask F, Kosaraju's algorithm exercise for finding SCC)

Run Kosaraju's algorithm on the following directed graph G and highlight the SCCs that you can find in that graph. Write down the discovery and finishing time of every vertex. Assume that neighbors are listed in increasing vertex label.

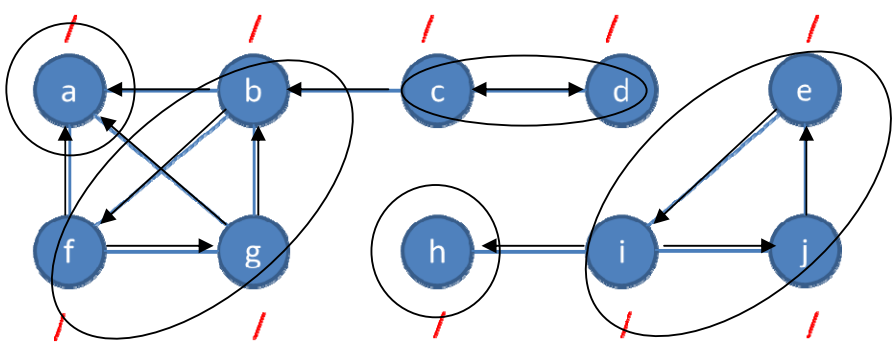


Solution:
The start and finishing times of each vertex (shown in the picture below) is as follows



Now if we traverse the vertices of G^T in decreasing finishing time, we have these 5 SCCs:

1. {h}, finishing time 20
2. {e,i,j}, finishing time 18
3. {a}, finishing time 12
4. {b,f,g}, finishing time 11
5. {c, d}, finishing time 6



Note that we can simplify the process of Kosaraju's algorithm as follows: Instead of saving f[a,b,c,d,e,f,g,i,j] = {12,11,6,5,18,9,10,20,16,17} and sort it in descending order, we can simply append the finished vertex to the back of a linked list of finished vertices f = {d, c, f, g, b, a, i, j, e, h}, and simply process this list from back to front, i.e. we avoid the need of sorting f as in Lecture 09 ☺.

D5. (Minor variant of Kosaraju's algorithm, is it still correct?)

In the lecture, we discussed a two-phase DFS algorithm to identify strongly connected components of a directed graph G. Suppose we replace the DFS algorithm in the second phase by BFS. Precisely, we change

Step 3: Call DFS(G^T) [Consider the vertices in order of decreasing f(u)]

into

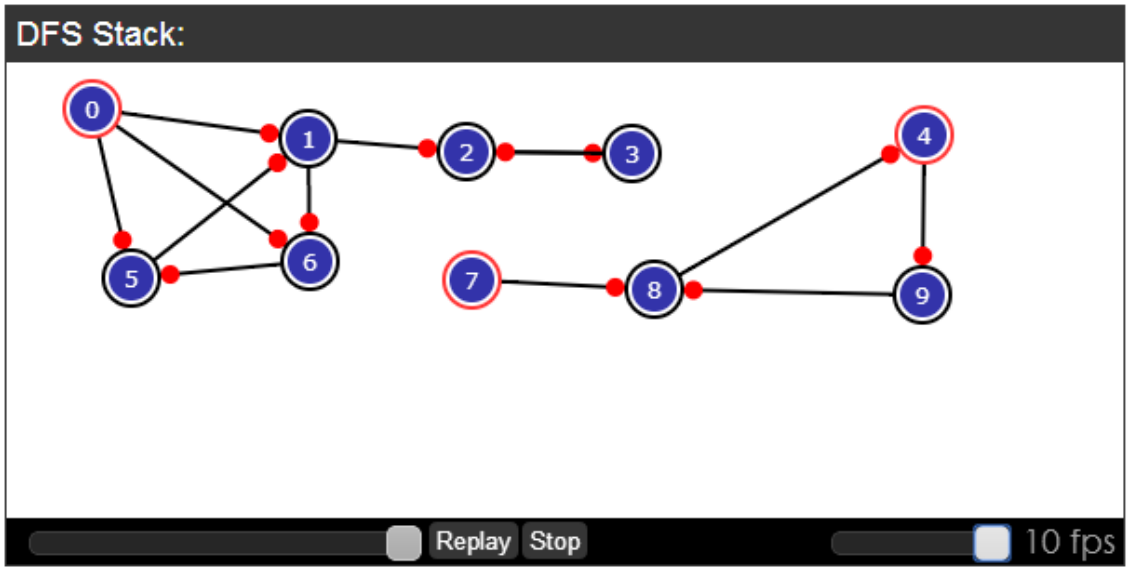
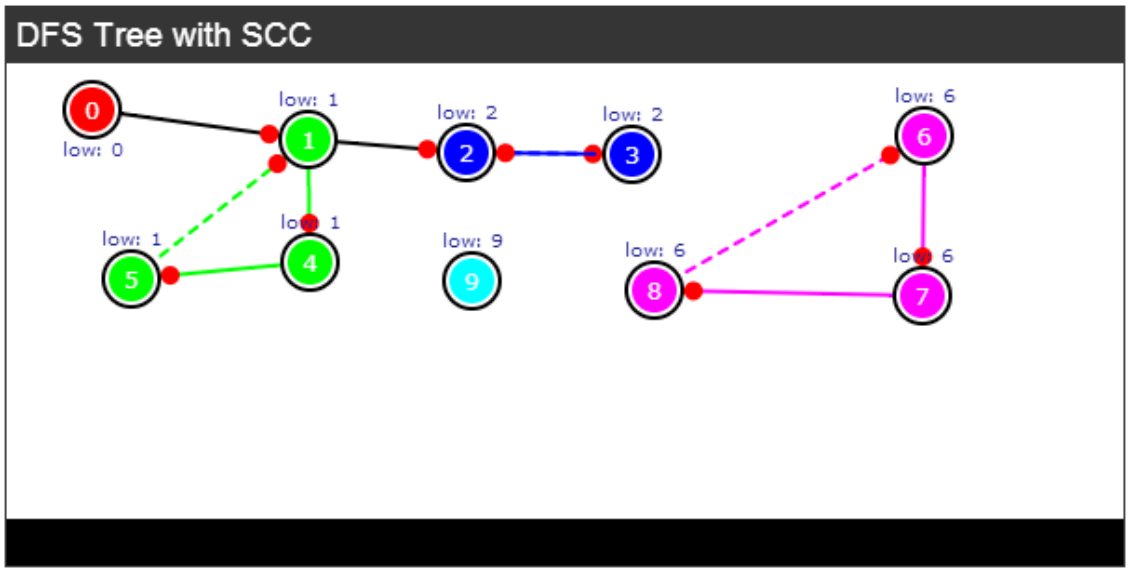
Step 3: Call BFS(G^T) [Consider the vertices in order of decreasing f(u)]

Can the modified Kosaraju's algorithm report the SCCs correctly?

Solution:

Yes. It is because both DFS/BFS traversal on G^T in decreasing f(u) will report the same reachable vertices as a strongly connected component. We only need DFS in the first step.

Advanced Problems-Try these for challenge and fun. There is no deadline for A-problems.
Turn in your attempts DIRECTLY to Steven. Do not combine it with your HW solutions.

Solve question D4 above with another algorithm that is not taught in CS3230 class:
Tarjan's algorithm (reference Competitive Programming 3, Section 4.2.9).
Note: All current/past CS3233 students are excluded from this coffee award.

The solution using visualization tool:
http://www.comp.nus.edu.sg/~stevenha/visualization/dfsbfs.html